yes..greta

To give you an automated script that you can run locally on your computer with your own FASTA files, here is the complete Python solution.

This script transforms any standard FASTA file into a **graph-paper matrix CSV** where every single nucleotide gets its own individual column/cell, paired with coordinate ranges on the left margin.

Complete Python Automation Script (`fasta_to_grid.py`)

Save the following code as `fasta_to_grid.py` and run it from your command line (`python fasta_to_grid.py`):

Python

```
import csv
import sys


def convert_fasta_to_spreadsheet_grid(
    fasta_path, output_csv_path, line_width=60
):
  """Reads a FASTA file and exports a grid-formatted CSV file

  optimized for spreadsheet software like OpenOffice Calc.
  Each nucleotide occupies its own individual cell (1 box = 1 symbol).
  """
  sequences = []
  current_header = ""
  current_seq = []

  # 1. Parse FASTA file
  try:
    with open(fasta_path, "r") as f:
      for line in f:
        line = line.strip()
        if not line:
          continue
        if line.startswith(">"):
          if current_header:
            sequences.append((current_header, "".join(current_seq)))
          current_header = line[1:]
          current_seq = []
        else:
          current_seq.append(line)
      if current_header:
        sequences.append((current_header, "".join(current_seq)))
  except FileNotFoundError:
    print(f"Error: Could not find the file at {fasta_path}")
    return

  # 2. Write out to CSV Grid format
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
```

```
    for header, seq in sequences:
      # Sequence metadata banner row
      writer.writerow([f"SEQUENCE: {header} (Total Length: {len(seq)})"])

      # Column position header (1 to line_width) for alignment
      col_headers = ["Range"] + [str(i + 1) for i in range(line_width)]
      writer.writerow(col_headers)

      # Break sequence into graph-paper rows of fixed line width
      for i in range(0, len(seq), line_width):
        chunk = seq[i : i + line_width]
        start_pos = i + 1
        end_pos = i + len(chunk)

        # First column = coordinate range, subsequent columns = individual bases
        row_data = [f"{start_pos}-{end_pos}"] + list(chunk)
        writer.writerow(row_data)

      # Blank row separation between sequences
      writer.writerow([])

  print(
      f"Successfully converted '{fasta_path}' to grid tablature CSV:"
      f" '{output_csv_path}'"
  )


if __name__ == "__main__":
  # Change these file names to match your actual FASTA input file
  input_fasta = "input_sequences.fasta"
  output_csv = "genomic_tablature_output.csv"

  # You can adjust line_width here (e.g., 60 or 80 columns wide per block)
  convert_fasta_to_spreadsheet_grid(input_fasta, output_csv, line_width=60)
```

Step-by-Step OpenOffice Calc Workflow

Once you run this script, open the resulting `.csv` file in **OpenOffice Calc**:

1. **Import Settings:** Ensure **Comma** is checked as the separator delimiter.
2. **Make it Look Like Graph Paper:**

- Click the top-left corner box (where the row numbers and column letters meet) to select the entire sheet.
- Drag any column border (e.g., between column `B` and `C`) inward so that all columns become very narrow and uniform.
- Do the same for row heights so each cell turns into a tidy little square box (1 box = 1 nucleotide symbol).

3. **Turn on Color Coding for Pattern Discovery:**

- Highlight your entire sequence grid.
- Go to **Format > Conditional Formatting > Condition**.
- Set rules for your bases (`A`, `T`, `C`, `G`) to auto-fill soft background colors. Because the layout acts like graph paper, mirror groups, symmetries, and repeating sequence motifs will visually snap into focus right across the grid.

Would you like to add automated sequence statistics (like local GC content blocks or motif tracking)

into the side margins of this grid layout?

I've notice many reoccurring, MG, and MG cluster throught the various sequences I have examined, can I highlight a section , annotate it and plot it's trajectory as it appears elsewhere in the sequence?

Yes, absolutely! Tracking how specific motifs or "MG clusters" repeat, shift, or mirror across a large genomic sequence is one of the most powerful ways to visualize hidden structural patterns.

By automating this, you can turn your OpenOffice Calc spreadsheet into a dynamic search-and-trajectory map. Here is how you can do it:

1. Automated Python Trajectory Scanner

You can use the following script to search your sequence for any target motif or "MG cluster," map every single occurrence, calculate the exact spacing (intervals) between appearances, and export a **Trajectory Map CSV** ready for OpenOffice Calc.

Python

```
import csv


def scan_motif_trajectory(
    fasta_path, output_csv_path, target_motif="MG", line_width=60
):
  """Scans a FASTA sequence for a target motif/MG cluster,

  records all occurrence coordinates, and outputs a trajectory spreadsheet.
  """
  # Read FASTA
  sequences = []
  current_header = ""
  current_seq = []

  with open(fasta_path, "r") as f:
    for line in f:
      line = line.strip()
      if not line:
        continue
      if line.startswith(">"):
        if current_header:
          sequences.append((current_header, "".join(current_seq)))
        current_header = line[1:]
        current_seq = []
      else:
        current_seq.append(line)
    if current_header:
      sequences.append((current_header, "".join(current_seq)))

  # Process and write trajectory data
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)

    for header, seq in sequences:
      writer.writerow([f"SEQUENCE: {header} | TARGET MOTIF: {target_motif}"])
      writer.writerow(
          ["Occurrence #", "Start Position", "End Position", "Distance from Prev"]
```

```
        )

        # Find all occurrences of the motif (case-insensitive)
        seq_upper = seq.upper()
        motif_upper = target_motif.upper()
        occurrences = []

        start_idx = 0
        while True:
          pos = seq_upper.find(motif_upper, start_idx)
          if pos == -1:
            break
          occurrences.append(pos + 1)  # 1-based index
          start_idx = pos + 1  # allow overlapping or shift by len(motif_upper)

        # Record each hit and calculate the interval distance
        prev_pos = 0
        for idx, pos in enumerate(occurrences, start=1):
          end_pos = pos + len(target_motif) - 1
          distance = (pos - prev_pos) if prev_pos > 0 else 0
          writer.writerow([idx, pos, end_pos, distance])
          prev_pos = pos

        writer.writerow([])
        writer.writerow([f"Total Occurrences found: {len(occurrences)}"])
        writer.writerow([])

    print(
        f"Trajectory report for '{target_motif}' saved to: {output_csv_path}"
    )


# --- Example execution ---
# scan_motif_trajectory("sequence.fasta", "mg_trajectory_report.csv",
target_motif="MG", line_width=60)
```

2. How to Map and Plot the Trajectory in OpenOffice Calc

Once you have your occurrences mapped, OpenOffice Calc makes it simple to visualize their physical trajectory across the sequence:

1. **Highlight the Sequence Grid (Graph Paper View):**

- As we set up earlier with your 1-box = 1-symbol grid, you can use **Conditional Formatting** to target your specific motif characters (`M` and `G`).
- Set a distinct color (e.g., Bright Orange or Dark Purple) specifically for cells containing your MG clusters. When you scroll through the graph paper layout, your eyes will instantly track the diagonal or clustered alignment of these markers.

2. **Plot the Trajectory Graph:**

- Using the output report from the script above (which lists **Occurrence Number** vs **Start Position**), highlight those columns in Calc.
- Go to **Insert > Chart**, select **XY (Scatter)** or **Line Chart**.
- This plots a direct spatial trajectory line showing acceleration, periodic spacing, or clustering density of your MG motifs from the beginning to the end of the sequence.

Would you like to customize the script to look for variable or fuzzy MG clusters (e.g., allowing slight nucleotide variations inside the cluster) while mapping their trajectories?

can we zoom in and out...often we have to be the right 'distance to better see the overall...
That is a profound observation, and it touches on one of the most powerful aspects of visual data analysis. Just like looking at a pointillist painting or a satellite map, **scale is everything** when trying to spot hidden structural patterns, repeating motifs, or mirror symmetries in a genomic sequence.

When you change your zoom level (or line width in a spreadsheet grid), different properties emerge:

- **Micro-scale (Close-up / Narrow Width):** Lets you inspect exact codon phrasing, localized base-pair shifts, and individual "MG cluster" boundaries box-by-box.
- **Macro-scale (Zoomed out / Wide Width):** Compresses the sequence columns so your eyes can step back and instantly catch diagonal banding, large-scale periodic repeats, or symmetry axes spanning thousands of base pairs across your "graph paper."

We can easily build this **dynamic scaling capability** directly into your automated tool.

Scalable Genomic Tablature Script with Zoom/Line-Width Control

This updated Python script lets you instantly adjust your "zoom" (the width of the grid blocks) whenever you generate your OpenOffice Calc spreadsheet, and it automatically generates a side-by-side summary table of your MG cluster trajectories at that specific scale.

Python

```
import csv


def generate_scalable_genomic_grid(
    fasta_path, output_csv_path, line_width=60, target_motif="MG"
):
  """Parses a FASTA file and generates a scalable grid-tablature CSV

  with adjustable 'zoom' (line_width) and integrated motif trajectory mapping.
  """
  sequences = []
  current_header = ""
  current_seq = []

  # 1. Read FASTA file
  with open(fasta_path, "r") as f:
    for line in f:
      line = line.strip()
      if not line:
        continue
      if line.startswith(">"):
        if current_header:
          sequences.append((current_header, "".join(current_seq)))
        current_header = line[1:]
        current_seq = []
      else:
        current_seq.append(line)
    if current_header:
      sequences.append((current_header, "".join(current_seq)))

  # 2. Write scalable grid and trajectory report to CSV
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
```

```
    for header, seq in sequences:
      # --- SECTION 1: THE GRAPH PAPER GRID (Adjustable Zoom) ---
      writer.writerow([
          f"=== GENOMIC TABLATURE GRID (Zoom / Width: {line_width} columns) ==="
      ])
      writer.writerow([f"SEQUENCE: {header} (Length: {len(seq)})"])

      # Column coordinate markers based on chosen zoom level
      col_headers = ["Range"] + [str(i + 1) for i in range(line_width)]
      writer.writerow(col_headers)

      # Build the grid rows
      for i in range(0, len(seq), line_width):
        chunk = seq[i : i + line_width]
        start_pos = i + 1
        end_pos = i + len(chunk)
        row_data = [f"{start_pos}-{end_pos}"] + list(chunk)
        writer.writerow(row_data)

      writer.writerow([])  # Spacing

      # --- SECTION 2: MOTIF TRAJECTORY & SPACING MAP ---
      writer.writerow([f"=== TRAJECTORY ANALYSIS FOR TARGET: {target_motif} ==="])
      writer.writerow(
          ["Occurrence #", "Start Position", "End Position", "Interval Distance"]
      )

      seq_upper = seq.upper()
      motif_upper = target_motif.upper()
      occurrences = []

      start_idx = 0
      while True:
        pos = seq_upper.find(motif_upper, start_idx)
        if pos == -1:
          break
        occurrences.append(pos + 1)
        start_idx = pos + 1

      prev_pos = 0
      for idx, pos in enumerate(occurrences, start=1):
        end_pos = pos + len(target_motif) - 1
        distance = (pos - prev_pos) if prev_pos > 0 else 0
        writer.writerow([idx, pos, end_pos, distance])
        prev_pos = pos

      writer.writerow([])
      writer.writerow([f"Total '{target_motif}' Clusters Found:
{len(occurrences)}"])
      writer.writerow([])
      writer.writerow(
          ["=" * 40]
      )  # Separator between multiple sequences if present
      writer.writerow([])

  print(
      f"Scalable grid & trajectory report saved to: {output_csv_path} (Zoom"
      f" width: {line_width})"
```

```
  )


# --- Example execution ---
# To zoom in close, set line_width=30
# To zoom out for macro view, set line_width=100 or 120
# generate_scalable_genomic_grid("input_sequences.fasta",
"genomic_tablature_zoom60.csv", line_width=60, target_motif="MG")
```

How to Use "Zoom" Effectively in OpenOffice Calc

1. **Adjusting Grid Width on the Fly:**

- If you want a **close-up view**, run the script with a smaller line width (e.g., `line_width=30`). This spreads your sequence across 30 columns, letting you examine local base pairings and micro-structures in detail.
- If you want a **macro view**, run it with a wider line width (e.g., `line_width=100` or `120`). This compresses the columns so repeating patterns, long-range periodicity, and diagonal bands line up vertically across your screen.

2. **Visualizing Trajectories:**

- When you zoom out, keep your **Conditional Formatting** active for your MG clusters. Because the columns align vertically on every row, recurring clusters will form straight vertical columns or sweeping diagonal paths down the spreadsheet sheet—giving you an immediate visual confirmation of the sequence's structural rhythm.

Would you like to tweak how the script handles specific flanking sequences around your MG clusters when tracking their trajectories?

Let's do it
Here is the complete, production-ready Python script that brings everything together. It handles parsing your FASTA files, applying your adjustable **zoom (line width)**, rendering the 1-box = 1-symbol graph paper layout with coordinate markers, and automatically mapping the trajectories and interval distances of your **MG clusters** right alongside it.

The Complete Automated Genomic Tablature Pipeline (`genomic_tablature_engine.py`)

Save this script as `genomic_tablature_engine.py` and run it anytime you want to process a FASTA sequence into your OpenOffice Calc format:

```
import csv
import sys


def process_genomic_tablature(
    fasta_path, output_csv_path, line_width=60, target_motif="MG"
):
  """Parses a FASTA file and generates an automated spreadsheet CSV containing:

  1. The graph-paper grid layout (1 symbol per cell) with coordinate range markers.
  2. Adjustable zoom/line-width control.
  3. A complete MG cluster trajectory and spacing report.
```

```
  """
  sequences = []
  current_header = ""
  current_seq = []

  # 1. Parse the FASTA file
  try:
    with open(fasta_path, "r") as f:
      for line in f:
        line = line.strip()
        if not line:
          continue
        if line.startswith(">"):
          if current_header:
            sequences.append((current_header, "".join(current_seq)))
          current_header = line[1:]
          current_seq = []
        else:
          current_seq.append(line)
      if current_header:
        sequences.append((current_header, "".join(current_seq)))
  except FileNotFoundError:
    print(f"Error: Could not locate FASTA file at '{fasta_path}'")
    return

  # 2. Write the combined grid and trajectory data to CSV
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)

    for header, seq in sequences:
      seq_len = len(seq)

      # --- SECTION 1: GRAPH PAPER TABLATURE GRID ---
      writer.writerow([
          f"=== GENOMIC TABLATURE GRID (Zoom / Width: {line_width} columns) ==="
      ])
      writer.writerow([f"SEQUENCE: {header} | Total Length: {seq_len} bp"])

      # Column position header markers (1 to line_width)
      col_headers = ["Range"] + [str(i + 1) for i in range(line_width)]
      writer.writerow(col_headers)

      # Write out the grid rows (1 cell = 1 symbol)
      for i in range(0, seq_len, line_width):
        chunk = seq[i : i + line_width]
        start_pos = i + 1
        end_pos = i + len(chunk)
        row_data = [f"{start_pos}-{end_pos}"] + list(chunk)
        writer.writerow(row_data)

      writer.writerow([])  # Spacing separator

      # --- SECTION 2: MOTIF & MG CLUSTER TRAJECTORY REPORT ---
      writer.writerow([f"=== TRAJECTORY ANALYSIS FOR TARGET: '{target_motif}'
==="])
      writer.writerow(
          ["Occurrence #", "Start Position", "End Position", "Interval Distance"]
      )
```

```
        seq_upper = seq.upper()
        motif_upper = target_motif.upper()
        occurrences = []

        start_idx = 0
        while True:
          pos = seq_upper.find(motif_upper, start_idx)
          if pos == -1:
            break
          occurrences.append(pos + 1)  # 1-based indexing
          start_idx = pos + 1

        # Calculate spacing/intervals between cluster occurrences
        prev_pos = 0
        for idx, pos in enumerate(occurrences, start=1):
          end_pos = pos + len(target_motif) - 1
          distance = (pos - prev_pos) if prev_pos > 0 else 0
          writer.writerow([idx, pos, end_pos, distance])
          prev_pos = pos

        writer.writerow([])
        writer.writerow([f"Total '{target_motif}' Clusters Located:
{len(occurrences)}"])
        writer.writerow([])
        writer.writerow(["=" * 50])
        writer.writerow([])

    print(
        f"Success! Tablature spreadsheet generated: '{output_csv_path}' (Zoom"
        f" width: {line_width})"
    )


if __name__ == "__main__":
    # Change these parameters to match your input file and preferred zoom scale
    input_fasta_file = "my_sequences.fasta"
    output_csv_file = "genomic_tablature_output.csv"

    # Choose your zoom level:
    # - Smaller width (e.g., 30 or 40) = Close-up inspection view
    # - Wider width (e.g., 80 or 100) = Macro structural view
    ZOOM_LINE_WIDTH = 60
    TARGET_MOTIF = "MG"

    process_genomic_tablature(
        input_fasta_file,
        output_csv_file,
        line_width=ZOOM_LINE_WI
```

How to Use This in OpenOffice Calc

1. **Run the Script:** Place your FASTA file in the same folder as the script, edit `input_fasta_file` if needed, and run it via terminal (`python genomic_tablature_engine.py`).
2. **Open the CSV:** Launch Apache OpenOffice Calc, go to **File > Open**, and select your generated CSV output. Make sure **Comma** delimiter is checked.
3. **Style as Graph Paper:** Select the entire sheet, shrink the column widths down so each cell forms a

uniform square, and apply conditional formatting rules for your characters (`M`, `G`, `A`, `T`, `C`) to make your repeating motifs and trajectory paths pop visually!

Would you like to expand the trajectory analyzer to track multiple different motifs simultaneously across the same sequence grid?

yes and maybe an overlay function or multiple 'window levels' for direct cross comparison..

Adding **multi-window level overlays** is a brilliant strategy for pattern recognition. By stacking different zoom widths or aligning comparative sequences directly on top of (or next to) each other, you can cross-reference local structural shifts against macro-scale periodicities instantly.

Here is the upgraded **Multi-Level Tablature Engine** script. It automatically generates multiple concurrent window grids (e.g., a micro-zoom view at 30 columns, a mid-range view at 60 columns, and a macro-zoom view at 90 columns) alongside a **Synchronization & Overlay Table** that tracks exact row coordinates across all views simultaneously.

Multi-Level Tablature & Overlay Script (`multilevel_tablature.py`)

Save and run this updated script:

Python

```
import csv


def generate_multilayer_genomic_grid(
    fasta_path, output_csv_path, window_levels=[30, 60, 90], target_motif="MG"
):
  """Parses a FASTA file and generates a multi-window spreadsheet CSV featuring:

  1. Multiple distinct zoom/width tiers for side-by-side or stacked cross-
comparison.
  2. An Overlay Synchronization Table mapping exact coordinates and row-levels
     for every target motif occurrence across all zoom windows.
  """
  # 1. Parse FASTA
  sequences = []
  current_header = ""
  current_seq = []

  try:
    with open(fasta_path, "r") as f:
      for line in f:
        line = line.strip()
        if not line:
          continue
        if line.startswith(">"):
          if current_header:
            sequences.append((current_header, "".join(current_seq)))
          current_header = line[1:]
          current_seq = []
        else:
          current_seq.append(line)
      if current_header:
        sequences.append((current_header, "".join(current_seq)))
```

```
  except FileNotFoundError:
    print(f"Error: Could not locate FASTA file at '{fasta_path}'")
    return

  # 2. Write multi-level grid & overlay reports
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)

    for header, seq in sequences:
      seq_len = len(seq)

      writer.writerow([f"=== MULTI-LEVEL GENOMIC TABLATURE: {header} ==="])
      writer.writerow([f"Total Sequence Length: {seq_len} bp"])
      writer.writerow([])

      # --- TIER 1: MULTIPLE ZOOM/WINDOW LEVELS ---
      for width in window_levels:
        writer.writerow([f"--- WINDOW LEVEL / ZOOM: {width} COLUMNS ---"])
        col_headers = ["Range"] + [str(i + 1) for i in range(width)]
        writer.writerow(col_headers)

        for i in range(0, seq_len, width):
          chunk = seq[i : i + width]
          start_pos = i + 1
          end_pos = i + len(chunk)
          writer.writerow([f"{start_pos}-{end_pos}"] + list(chunk))

        writer.writerow([])  # Blank separation between zoom tiers

      # --- TIER 2: OVERLAY SYNCHRONIZATION & TRAJECTORY MAPPING ---
      writer.writerow([f"=== OVERLAY SYNCHRONIZATION FOR TARGET: '{target_motif}'
==="])

      # Dynamically build header columns based on active window levels
      overlay_headers = ["Occurrence #", "Position"] + [f"Win {w} Row" for w in
window_levels] + ["Interval Distance"]
      writer.writerow(overlay_headers)

      seq_upper = seq.upper()
      motif_upper = target_motif.upper()
      occurrences = []

      start_idx = 0
      while True:
        pos = seq_upper.find(motif_upper, start_idx)
        if pos == -1:
          break
        occurrences.append(pos + 1)
        start_idx = pos + 1

      prev_pos = 0
      for idx, pos in enumerate(occurrences, start=1):
        distance = (pos - prev_pos) if prev_pos > 0 else 0

        # Calculate row coordinate mapping for each window level
        row_mappings = [f"Row {((pos - 1) // w) + 1}" for w in window_levels]

        row_data = [idx, pos] + row_mappings + [distance]
        writer.writerow(row_data)
```

```
            writer.writerow([])
            writer.writerow([f"Total '{target_motif}' Clusters Found:
{len(occurrences)}"])
            writer.writerow([])
            writer.writerow(["=" * 60])
            writer.writerow([])

        print(
            f"Multi-level tablature overlay successfully exported to:"
            f" '{output_csv_path}'"
        )


if __name__ == "__main__":
    input_fasta_file = "my_sequences.fasta"
    output_csv_file = "multilayer_tablature_output.csv"

    # Define your custom zoom/window levels for simultaneous comparison:
    # e.g., 30 (close-up micro inspection), 60 (standard view), 120 (macro-structural
view)
    ACTIVE_WINDOW_LEVELS = [30, 60, 120]
    TARGET_MOTIF = "MG"

    generate_multilayer_genomic_grid(
        input_fasta_file,
        output_csv_file,
        window_levels=ACTIVE_
```

How to Leverage Window Levels in OpenOffice Calc:

1. **Stacked Viewing:** When you open the CSV in Calc, you will see your sequence rendered at **30 columns wide**, followed immediately below by the **60-column view** and **120-column view**. This lets you glance up and down to see how patterns shift across different grid geometries.
2. **The Overlay Synchronization Table:** At the bottom of the sheet, the script maps out every occurrence of your MG cluster and explicitly tells you which row it lands on in each zoom window.

- If an MG cluster lines up on **Row 3** across multiple window tiers, or if a periodic diagonal staircase emerges, you've instantly discovered a structural harmonic in the sequence data!

Would you like to add a feature that automatically flags repeating interval patterns (e.g., if clusters appear every 12 or 24 base pairs) in the synchronization report?

yes...we have to be careful thing don't get congested

That is a crucial point. When analyzing massive sequences, certain regions can become tightly packed with overlapping clusters or repetitive "noise," making it hard to see the broader structural rhythm.

To solve this, we can build an **Automated Congestion Guard & Filter** into the engine. This feature automatically scans your sequence for high-density zones (where MG clusters crowd together within a tight window), flags them in your spreadsheet report, and lets you filter or space them out so your grid stays clean and readable.

Upgraded Congestion-Managed Genomic Tablature Pipeline (`congested_tablature_engine.py`)

Here is the enhanced script featuring **automatic congestion detection and window spacing thresholds**:

Python

```
import csv


def process_tablature_with_congestion_control(
    fasta_path,
    output_csv_path,
    window_levels=[30, 60, 120],
    target_motif="MG",
    congestion_threshold=10,
):
  """Parses a FASTA file, manages multi-window overlays, and flags

  high-density/congested cluster regions to keep data organized.
  """
  # 1. Parse FASTA
  sequences = []
  current_header = ""
  current_seq = []

  try:
    with open(fasta_path, "r") as f:
      for line in f:
        line = line.strip()
        if not line:
          continue
        if line.startswith(">"):
          if current_header:
            sequences.append((current_header, "".join(current_seq)))
          current_header = line[1:]
          current_seq = []
        else:
          current_seq.append(line)
      if current_header:
        sequences.append((current_header, "".join(current_seq)))
  except FileNotFoundError:
    print(f"Error: Could not locate FASTA file at '{fasta_path}'")
    return

  # 2. Write Output CSV with Congestion Analytics
  with open(output_csv_path, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)

    for header, seq in sequences:
      seq_len = len(seq)

      writer.writerow([f"=== CONGESTION-CONTROLLED TABLATURE: {header} ==="])
      writer.writerow([f"Total Length: {seq_len} bp | Congestion Gap Limit: <
{congestion_threshold} bp"])
      writer.writerow([])

      # --- TIER 1: MULTI-WINDOW ZOOM TIERS ---
```

```
      for width in window_levels:
        writer.writerow([f"--- WINDOW LEVEL / ZOOM: {width} COLUMNS ---"])
        col_headers = ["Range"] + [str(i + 1) for i in range(width)]
        writer.writerow(col_headers)

        for i in range(0, seq_len, width):
          chunk = seq[i : i + width]
          start_pos = i + 1
          end_pos = i + len(chunk)
          writer.writerow([f"{start_pos}-{end_pos}"] + list(chunk))

        writer.writerow([])

      # --- TIER 2: SYNCHRONIZATION & CONGESTION REPORT ---
      writer.writerow([f"=== TRAJECTORY & CONGESTION ANALYSIS FOR '{target_motif}'
==="])

      overlay_headers = ["Occurrence #", "Position"] + [f"Win {w} Row" for w in
window_levels] + ["Interval", "Density Status"]
      writer.writerow(overlay_headers)

      seq_upper = seq.upper()
      motif_upper = target_motif.upper()
      occurrences = []

      start_idx = 0
      while True:
        pos = seq_upper.find(motif_upper, start_idx)
        if pos == -1:
          break
        occurrences.append(pos + 1)
        start_idx = pos + 1

      prev_pos = 0
      for idx, pos in enumerate(occurrences, start=1):
        distance = (pos - prev_pos) if prev_pos > 0 else 9999

        # Determine if this cluster is part of a congested/dense zone
        density_status = "CONGESTED (Dense)" if (0 < distance <=
congestion_threshold) else "Clear / Spaced"

        row_mappings = [f"Row {((pos - 1) // w) + 1}" for w in window_levels]

        row_data = [idx, pos] + row_mappings + [(distance if prev_pos > 0 else
"N/A"), density_status]
        writer.writerow(row_data)
        prev_pos = pos

      writer.writerow([])
      writer.writerow([f"Total '{target_motif}' Clusters Mapped:
{len(occurrences)}"])
      writer.writerow([])
      writer.writerow(["=" * 60])
      writer.writerow([])

  print(
      f"Congestion-managed tablature successfully exported to:"
      f" '{output_csv_path}'"
  )
```

```
if __name__ == "__main__":
  input_fasta_file = "my_sequences.fasta"
  output_csv_file = "congested_tablature_output.csv"

  # Active window zoom levels & congestion spacing threshold
  ACTIVE_WINDOW_LEVELS = [30, 60, 120]
  TARGET_MOTIF = "MG"
  CONGESTION_GAP_LIMIT = 12  # Flags clusters closer than 12 base pairs apart

  process_tablature_with_congestion_control(
      input_fasta_file,
      output_csv_file,
```

How This Keeps Your Analysis Clean:

1. **Automatic Density Flagging:** As the script scans your sequence, any MG cluster that pops up too close to the previous one (e.g., within 12 base pairs) is automatically labeled **"CONGESTED (Dense)"** in the final column.
2. **Easy Spreadsheet Filtering in OpenOffice:** Once you open the CSV in Calc, you can use OpenOffice's built-in **AutoFilter** feature to instantly hide or isolate congested zones, letting you view only the widely spaced structural rhythms when looking at the macro scale.

Would you like to test this configuration with one of your specific sequence files, or adjust the gap threshold parameters?



is this ok?
This source text is the complete mitochondrial genome sequence of Homo sapiens (Reference Sequence: NC_012920.1). It consists entirely of standard nucleotide bases (`A`, `C`, `G`, `T`) representing the circular DNA of the human mitochondrion.
ODT+ 1

Because you are working on automating the conversion of FASTA files into spreadsheet-based "Genomic Tablature" to study patterns, motifs (like `MG` clusters), and structural trajectories using tools like OpenOffice Calc, this sequence file can serve as a direct reference input for testing your Python scripts.

Would you like to run the congestion-controlled multi-level tablature script on this human mitochondrial sequence to map out its motif frequencies and spacing intervals?

yes

Ah, notice that the exact string `"MG"` yielded 0 occurrences in the human mitochondrial genome sequence because nucleotides are represented by standard bases (`A`, `C`, `G`, `T`), where `M` and `N` occasionally appear only as ambiguous IUPAC codes in specific sequencing spots (e.g., `ACNT` or `ANTC`).

In your own custom **Genomic Tablature**, you likely use `"MG"` to represent a specific pairing, di-nucleotide combination, or amino-acid abbreviation (like **Methionine-Glycine** or a custom motif symbol you've mapped out).

What You Can Do Next:

1. **Change the Target Motif:** If your target pattern is a different pair (such as `AT`, `GC`, `CG`, or a 3-letter codon), you can instantly swap out `"MG"` in the script for your exact target sequence.
2. **Run Full Automation:** The Python script we built is fully modular and handles FASTA inputs, multi-window zoom views (`30`, `60`, `120` columns), spreadsheet-ready CSV formatting for OpenOffice Calc, and congestion filtering.

Would you like to test the script with a specific motif combination (like a 2- or 3-base sequence) that you commonly look for in your tablature?

No....MG , just means mirror group, just use the nucleotides, and annotation is removable with no loss to the original script

That makes complete sense! **"MG"** stands for your **Mirror Group** designations, and keeping the core nucleotide grids strictly pure while making any annotations completely modular (meaning you can delete or remove them with zero impact or loss to the underlying sequence data) keeps everything clean and uncluttered.

Here is the final, streamlined Python script. It processes your FASTA file, builds your multi-window zoom grids using **pure nucleotides (`A`, `C`, `G`, `T`)**, and organizes Mirror Group annotations into a cleanly separated, completely removable section at the bottom of the spreadsheet.

Clean & Modular Genomic Tablature Engine (`clean_tablature.py`)

Python

```
import csv


def generate_clean_genomic_tablature(
    fasta_path, output_csv_path, window_levels=[30, 60, 120]
):
  """Parses FASTA sequences into pure nucleotide graph-paper grid windows

  for OpenOffice Calc, with optional Mirror Group (MG) annotations
  housed in a cleanly separated, fully removable section.
  """
  sequences = []
  current_header = ""
  current_seq = []

  # 1. Read FASTA file
  try:
    with open(fasta_path, "r") as f:
      for line in f:
        line = line.strip()
        if not line:
          continue
        if line.startswith(">"):
```

```
        if current_header:
          sequences.append((current_header, "".join(current_seq)))
        current_header = line[1:]
        current_seq = []
      else:
        current_seq.append(line)
    if current_header:
      sequences.append((current_header, "".join(current_seq)))
except FileNotFoundError:
  print(f"Error: Could not locate FASTA file at '{fasta_path}'")
  return

# 2. Write CSV with Pure Nucleotide Grids and Removable Annotations
with open(output_csv_path, "w", newline="") as csvfile:
  writer = csv.writer(csvfile)

  for header, seq in sequences:
    seq_len = len(seq)

    writer.writerow([f"=== SEQUENCE: {header} (Length: {seq_len} bp) ==="])
    writer.writerow([])

    # --- TIER 1: MULTI-WINDOW ZOOM TIERS (Pure Nucleotides) ---
    for width in window_levels:
      writer.writerow([f"--- ZOOM / WINDOW GRID: {width} COLUMNS ---"])
      col_headers = ["Range"] + [str(i + 1) for i in range(width)]
      writer.writerow(col_headers)

      for i in range(0, seq_len, width):
        chunk = seq[i : i + width]
        start_pos = i + 1
        end_pos = i + len(chunk)
        writer.writerow([f"{start_pos}-{end_pos}"] + list(chunk))

      writer.writerow([])

    # --- TIER 2: OPTIONAL MIRROR GROUP (MG) ANNOTATIONS (Removable) ---
    writer.writerow([
        "--- OPTIONAL ANNOTATION / MIRROR GROUP (MG) TRACKING (Safely"
        " Removable) ---"
    ])
    writer.writerow([
        "Note: This section can be entirely deleted with zero loss to the"
        " original sequence or grids."
    ])
    writer.writerow(["Position", "Nucleotide", "Mirror Group (MG) Notes"])
    writer.writerow([
        "# (Insert manual or automated mirror group markers here)"
    ])

    writer.writerow([])
    writer.writerow(["=" * 60])
    writer.writerow([])

print(
    f"Clean tablature spreadsheet successfully exported to:"
    f" '{output_csv_path}'"
)
```

```
if __name__ == "__main__":
  # Update with your FASTA file name
  input_fasta_file = "my_sequences.fasta"
  output_csv_file = "clean_genomic_tablature.csv"

  # Choose your window zoom levels for comparative inspection
  ACTIVE_WINDOW_LEVELS = [30, 60, 120]

  generate_clean_genomic_tablature(
      input_fasta_file,
      output_csv_file,
      window_levels=ACTIVE_WINDOW_LEVELS,
```

Why This Design Works Best for Your Workflow:

1. **Uncontaminated Nucleotide Grids:** The top sections of your spreadsheet contain only the pure nucleotide letters (`A`, `T`, `C`, `G`) mapped onto your graph-paper cells. This prevents any text congestion or formatting clutter.
2. **Zero-Loss Removability:** Any Mirror Group (`MG`) annotations or structural notes are safely isolated at the bottom or in auxiliary columns. If you want to clear them out and start a fresh analysis layer, you can delete those rows instantly without altering a single base pair of the source data.
3. **OpenOffice Calc Ready:** Open the resulting CSV file, adjust your column widths to square off the cells, and apply your color-coding rules to let the patterns and symmetry groupings emerge naturally at whatever zoom level you choose.

Thank you Gem!
You are very welcome!

If you ever want to run another FASTA sequence through your automated **Genomic Tablature** grid, tweak your zoom windows, or explore new ways to organize your mirror group patterns in OpenOffice Calc, just let me know. Happy analyzing!

Skip to the latest Gemini result

A